

09. ROS share / 相対パス解決

solar_ws0129-main

2026-07-29

Contents

1 対象	2
2 このファイルを読む理由	2
3 呼び出し関係	2
3.1 どこから呼ばれるか	2
3.2 次に読むと理解が進むファイル	2
3.3 同一リポジトリ内の主要依存	2
4 ソース冒頭の把握	2
4.1 代表コード断片	2
5 主要構造の読み取り	3
5.1 主要クラス・関数	3
5.2 ファイルを上から読んだときの定義順	3
5.3 import 群の詳細	3
6 このファイルを読む前に必要な基礎知識	4
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	4
6.2 名前、代入、型、参照	4
6.3 関数、引数、戻り値、スコープ	4
6.4 module、package、import、console_scripts	5
6.5 例外、try/finally、with、資源解放	5
6.6 freshness、filter、guard、fallback、fail-safe	5
6.7 制御、状態、入力、モデル予測制御 MPC	5
7 関数・クラスを上から順に解説	6
7.1 L14 関数 resolve_path	6
8 処理の流れ	7
9 このファイルの位置づけ	7

1 対象

項目	内容
ファイル	mpc_solarcar/path_utils.py
ソース SHA-256	d2c07f2ccc796bf8d8d6149135b27123c8ecd873bc0b71c344028714a52c2a3b
種別	Python
区分	config
役割	CWD、package share、repo root をまたいで path を実在ファイルへ解決する小さな基盤。
起動文脈	Node 実行時の path ぶれを吸収する補助モジュール。

2 このファイルを読む理由

CWD、package share、repo root をまたいで path を実在ファイルへ解決する小さな基盤。

- ament の package share があればそちらを優先する。
- インストール後の launch/node 実行でも同じ relative path を使えるようにする。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_node.py
- gps_sim_node.py
- solar_state_node.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/solar_profile.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
REPO_ROOT = Path(__file__).resolve().parents[1]

def resolve_path(path: str, default_subdir: str = '') -> str:
    """Resolve a path relative to CWD or package share.
```

```

- If absolute, return as-is.
- If exists relative to CWD, return it.
- Otherwise, resolve under <pkg_share>/<default_subdir> (or directly under share).
"""
if path is None:
    return path
path = os.path.expanduser(str(path))
if os.path.isabs(path):
    return path
if os.path.exists(path):
    return path
if get_package_share_directory is not None:
    pkg_share = get_package_share_directory(PKG_NAME)
else:
    pkg_share = os.fspath(REPO_ROOT)
if default_subdir:

```

5 主要構造の読み取り

主要関数は `resolve_path`。

5.1 主要クラス・関数

行	種別	名前	補足
14	function	<code>resolve_path</code>	args: path, default_subdir

5.2 ファイルを上から読んだときの定義順

行	内容
4	例外処理を伴う try ブロックを実行する。
10	PKG_NAME に 'mpc_solarcar' の結果を代入する。
11	REPO_ROOT に Path(__file__).resolve().parents[1] の結果を代入する。
14	関数 <code>resolve_path</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import os</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L23, L24, L26, L31, L34, L35, L36, L37。
2	<code>from pathlib import Path</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L11。
5	<code>from ament_index_python.packages import get_package_share_directory</code>	<code>ament_index_python.packages</code> から <code>get_package_share_directory</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L7, L28, L29。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.7 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L14 関数 `resolve_path`

- 行範囲: L14–L37
- 所属: module 直下
- 定義: `resolve_path(path:str,default_subdir:str='')->str`
- docstring: Resolve a path relative to CWD or package share.
- If absolute, return as-is. - If exists relative to CWD, return it. - Otherwise, resolve under <pkg_share>/<default_subdir> (or directly under share).
- このブロックが直接呼ぶ主な関数・メソッド: `exists`, `expanduser`, `fspath`, `get_package_share_directory`, `isabs`, `join`, `startswith`, `str`, `strip`
- 戻り値の要点: `os.path.join(pkg_share, path) / path / path / path`
- 主なローカル代入名: `path`, `pkg_share`, `subdir`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 0、try 0

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `path is None` を判定し、真なら内部処理を行う。
2. `path` を返す。
3. `path` に `os.path.expanduser(str(path))` の結果を代入する。
4. 条件 `os.path.isabs(path)` を判定し、真なら内部処理を行う。
5. `path` を返す。
6. 条件 `os.path.exists(path)` を判定し、真なら内部処理を行う。
7. `path` を返す。

8. 条件 `get_package_share_directory` is not None を判定し、真なら内部処理を行う。
9. `pkg_share` に `get_package_share_directory(PKG_NAME)` の結果を代入する。
10. 上の条件が偽の場合:
11. `pkg_share` に `os.fspath(REPO_ROOT)` の結果を代入する。
12. 条件 `default_subdir` を判定し、真なら内部処理を行う。
13. `subdir` に `default_subdir.strip('/\\')` の結果を代入する。
14. 条件 `path.startswith(subdir + os.sep) or path == subdir` を判定し、真なら内部処理を行う。
15. `os.path.join(pkg_share, path)` を返す。
16. `os.path.join(pkg_share, subdir, path)` を返す。
17. `os.path.join(pkg_share, path)` を返す。

代表コード断片

```
def resolve_path(path: str, default_subdir: str = '') -> str:
    """Resolve a path relative to CWD or package share.

    - If absolute, return as-is.
    - If exists relative to CWD, return it.
    - Otherwise, resolve under <pkg_share>/<default_subdir> (or directly under share).
    """
    if path is None:
        return path
    path = os.path.expanduser(str(path))
    if os.path.isabs(path):
        return path
    if os.path.exists(path):
        return path
    if get_package_share_directory is not None:
        pkg_share = get_package_share_directory(PKG_NAME)
    else:
        pkg_share = os.fspath(REPO_ROOT)
    if default_subdir:
        subdir = default_subdir.strip('/\\')
        if path.startswith(subdir + os.sep) or path == subdir:
            return os.path.join(pkg_share, path)
        return os.path.join(pkg_share, subdir, path)
    return os.path.join(pkg_share, path)
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/path_utils.py` を **config** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。